OPEN TALK · 2026

Design *Thinking*

From theory to practice — an open discussion with student groups and faculty on design thinking in modern software engineering.

WHAT IS DESIGN THINKING?

DT IN REQUIREMENT ENGINEERING

DT IN INTENSIVE SOFTWARE DEV

QUESTION 01

What is *Design Thinking?*

A **human-centered problem-solving approach** — focused on deeply understanding user needs, reframing problems, and generating creative solutions through **rapid, iterative experimentation**. Not a rigid process — but a *flexible mindset*.

Human-centered

Starts from the **real needs** of users — not from technology assumptions or team biases. Empathy is the foundation.

Iterative

Test early, fail fast, learn continuously. Each loop refines the team's **understanding of the actual problem**.

Collaborative

Combines diverse perspectives: engineering, design, business, end users — **cross-functional thinking** in one room.

Action-oriented

Turn ideas into **tangible prototypes** fast. "Show, don't tell" — bias toward action over endless analysis.

FRAMEWORK

The 5 Stages of *Design Thinking*

1 — Empathize
Observe and listen to users — no judgment, no assumptions
User interview · Shadowing

→

2 — Define
Synthesize insights into a clear Point-of-View about the core problem
HMW · POV Statement

→

3 — Ideate
Unconstrained brainstorming — diverge first, converge later
Crazy 8 · Mind Map

→

4 — Prototype
Build fast, build cheap — good enough to learn from, not to ship
Paper · Wireframe · PoC

→

5 — Test
Put prototypes in front of real users — observe, capture, iterate
Usability Test · A/B

↻ Non-linear — insights from Test can send the team back to Empathize or Define. Iteration is a feature, not a failure.

MINDSET SHIFT

A different way of thinking vs. traditional approaches

TRADITIONAL / WATERFALL

- Requirements gathering
- System Analysis
- Design Document
- Development
- Testing
- Deployment

VS

DESIGN THINKING

- Empathize — understand the user
- Define — frame the right problem
- Ideate — expand solution space
- Prototype — make it tangible fast
- Test — learn and loop back

↺ Can return to any stage at any time

> *Waterfall excels when* **requirements are stable and well-understood upfront**. *Design Thinking shines when* **the problem is ambiguous and users don't yet know what they need** *— especially during discovery and requirements definition.*

QUESTION 02

Design Thinking in *Requirement Engineering*

Traditional RE asks: **"What does the system need to do?"**
Design Thinking shifts the question to: **"What do users actually need — and why?"** Then the system gets defined.

Collecting requirements from stakeholders	→	**Uncovering hidden needs through observation**
Writing hundred-page SRS documents	→	**Lightweight prototypes to validate early**
Modeling actors and use cases	→	**Empathy mapping + User journey**
Freezing requirements at project start	→	**Iterative refinement across sprints**

DT TOOL IN RE

Empathy Map — uncovering what users really need

SAYS 🗣

- "I just need the system to be faster"
- "This feature is unnecessary"
- "I don't understand why we have to do it this way"

THINKS 💭

- "Why is this workflow so complicated?"
- "I'm afraid of making mistakes with the system"
- "The IT team doesn't understand my actual job"

DOES ✋

- "Runs Excel in parallel with the system"
- "Manually copy-pastes data every single day"
- "Skips features they don't understand how to use"

FEELS 💛

- "Frustrated that the system doesn't match their workflow"
- "Anxious when using new features"
- "Satisfied only when they find personal shortcuts"

👤 *User: Accountant — age 35, uses the system 3h/day*

> *The Empathy Map surfaces the **gap** between what users say and what they actually think / feel / do — hidden requirements that never appear in traditional interviews.*

IN PRACTICE

4 DT Techniques in Requirement Engineering

🗺 Empathy Mapping

4 quadrants: Says / Thinks / Does / Feels — surfaces contradictions between what users say and what they actually do.

> *Ex: User says "I need better reports" — they actually just need a fast Excel export.*

→ EMPATHIZE STAGE

💡 HMW Questions

"How Might We…" reframes problems as design opportunities — expands the solution space instead of prematurely narrowing it.

> *Ex: "HMW help accountants stop copy-pasting data manually every day?"*

→ DEFINE STAGE

📋 User Story Mapping

Lay out the user journey horizontally, tasks vertically — see the backlog by delivered value, not just effort.

> *Ex: Discovered 3 independent tasks that could release MVP 2 sprints earlier.*

→ DEFINE + IDEATE

📄 Paper Prototyping

Validate requirements with hand-drawn sketches before writing code — catch UX and flow errors at near-zero cost.

> *Ex: A 30-min sketch session caught a login flow confusion → saved 2 sprints of refactoring.*

→ PROTOTYPE + TEST

QUESTION 03

DT in *Intensive Software* Development Projects

WHAT IS INTENSIVE SOFTWARE DEVELOPMENT?

Projects with **tight deadlines**, large teams, constantly shifting scope, and pressure from all directions. This environment leads many teams to skip DT — and that is the classic mistake. DT is not extra work — it's a **different way of thinking that ensures you build the right thing from the start**.

- **Fail fast, learn faster:** Small prototypes expose mistakes in week 1, not sprint 10. The cost to fix issues early is up to 100× lower than fixing them late.
- **Shared understanding beats long specs:** A 2-hour DT workshop creates better alignment than a 50-page requirements document — everyone sees the same problem.
- **User-centered backlog:** Every user story grounded in real insight — reduces "build the wrong thing" mistakes that waste future sprints on rework.
- **Creative confidence:** DT practice builds a culture of "try instead of overthink" — perfectly suited to the pace of intensive projects.

REAL INTEGRATION

DT integrated into *the Sprint Cycle*

SPRINT 0
DT Foundation

- **Empathy Workshop** — interview & observe users (2–4h)
- **Empathy Maps** — synthesize insights as a team
- **HMW session** — reframe problems as opportunities
- **User Story Map** — build a journey-anchored backlog

SPRINT PLANNING
Every sprint

- **Paper prototype** each story before assigning
- **5-minute sketch** to align team on UI/flow
- Refine acceptance criteria based on prototype
- Drop stories with no grounding in user insight

SPRINT REVIEW
End of sprint

- **Demo with real users**, not just stakeholders
- Observe actual usage behavior — don't just ask opinions
- Capture new insights for the next sprint
- Update Empathy Maps with new findings

RETROSPECTIVE
After sprint

- **"What did we learn about users?"** not just process
- Review which assumptions proved right or wrong
- Adjust story map if the user journey has shifted
- Identify DT activities needed next sprint

*DT doesn't replace Agile — it **amplifies** the quality of each sprint's input by ensuring the team always solves the right problem for the right users.*

ASSESSMENT

Benefits & Real-World Challenges

✦ MEASURABLE BENEFITS

- ✓ **Early risk detection** — week-1 prototypes replace month-1 specs; problems surface before a line of code is written.
- ✓ **Dramatically less rework** — continuous validation ensures the team builds what's actually needed, not what's assumed.
- ✓ **Faster team alignment** — shared empathy creates a common language across dev, BA, and business stakeholders.
- ✓ **Innovation culture** — DT mindset builds a culture of experimentation and reduces fear of failure within teams.

◆ PRACTICAL CHALLENGES

- ! **Upfront time cost** — Empathize and Define phases require investment before any tangible output appears.
- ! **Hard to scale** — DT workshops work best with teams of 5–9; 20+ people need an experienced facilitator.
- ! **Mindset shift takes time** — "solution-first" culture often resists "problem-first" thinking at the start.
- ! **Deadline pressure** — under squeeze, DT activities are first to get cut — requires strong management sponsorship.

> 💡 **When to apply DT?** — Most effective when the problem is ambiguous, users don't yet know what they need, or the current solution has high friction. Less suited for well-defined technical tasks with stable, clear requirements.

TAKEAWAYS

3 things to remember from today's open talk

01

DT is a mindset, not a process

You don't need to follow all 5 steps. What matters is **starting from the user**, not from the solution. Even spending 30 minutes observing users before coding is a DT mindset in action.

02

DT and Agile complement each other

DT answers "build the right thing" — Agile answers "build the thing right". **Sprint 0 + DT activities** is the most effective integration point for intensive projects.

03

Early prototypes save the most

The cost of fixing bugs grows **exponentially over time**. A 30-minute paper prototype saves more than 2 sprints of refactoring. Fail fast, learn faster — not a motto, a strategy.

OPEN DISCUSSION

Discussion
for groups & faculty

01 In your group's project, *which stage of Design Thinking* was the hardest to apply — and how did you handle it under deadline pressure?

02 Traditional RE vs. DT-based RE — *when should you use which?* In a real project, how do you know when you've done enough Empathizing to move to Define?

03 How do you *convince management* to invest in Sprint 0 DT activities when the pressure to "start coding immediately" is overwhelming?

Do faculty or other groups have additional perspectives or real-world examples to share?